Guide to Creating Custom Filters

This guide explains how to create custom filter classes for your domain entities using the SqlFilter interface and the GenericSqlFilter utility. This approach allows you to build dynamic and reusable SQL WHERE clauses for your database queries.

1. Overview

The filtering system relies on two main components:

1. **SqlFilter Interface**: A functional interface that any filter class must implement. It has a single method toSql() which returns a SQL string and a list of parameters.
2. **GenericSqlFilter Utility**: A helper class that implements the Composite Pattern to build complex queries safely (handling AND, OR, LIKE, =, >, <, etc.) and manages parameterized values to prevent SQL injection.

2. Implementing a New Filter

To create a filter for a specific entity (e.g., Product, Order, Employee), follow these steps:

Step 1: Create the Filter Class

Create a class that implements SqlFilter. Add fields representing the criteria you want to filter by (e.g., name, minPrice, status).

Example: ProductFilter.java

```
import org.example.paginarefiltraredb.domain.filters.SqlFilter;
import org.example.paginarefiltraredb.domain.filters.GenericSqlFilter;
import org.example.paginarefiltraredb.repository.paging.util.Pair;
import java.util.List;

public class ProductFilter implements SqlFilter {

    // 1. Define Filter Fields
    private String nameSearch;
    private Double minPrice;
    private Double maxPrice;
    private String category;

    // 2. Add Setters (Used by the UI Controller to bind input values)
    public void setNameSearch(String nameSearch) { this.nameSearch = nameSearch; }
    public void setMinPrice(Double minPrice) { this.minPrice = minPrice; }
    public void setMaxPrice(Double maxPrice) { this.maxPrice = maxPrice; }
```

```
    public void setCategory(String category) { this.category = category; }

    // 3. Implement toSql() using GenericSqlFilter
    @Override
    public Pair<String, List<Object>> toSql() {
        return GenericSqlFilter.and()
                .like("name", nameSearch)       // Maps to SQL: name LIKE ?
                .gt("price", minPrice)          // Maps to SQL: price > ?
                .lt("price", maxPrice)          // Maps to SQL: price < ?
                .eq("category_type", category)  // Maps to SQL: category_type = ?
                .toSql();
    }
}
```

Step 2: Understanding GenericSqlFilter Methods

The GenericSqlFilter class provides fluent methods to build your query.

- **GenericSqlFilter.and()**: Starts a new filter group where all conditions are joined by AND.
- **GenericSqlFilter.or()**: Starts a new filter group where all conditions are joined by OR.
- **.eq("column_name", value)**: Adds an equality condition (col = ?). *Note: If value is null, this condition is skipped automatically.*
- **.like("column_name", value)**: Adds a LIKE condition (col LIKE ?). *Note: You typically need to add wildcards % to the value before passing it, or handle it in your service.*
- **.gt("column_name", value)**: Adds a greater-than condition (col > ?).
- **.lt("column_name", value)**: Adds a less-than condition (col < ?).
- **.add(otherFilter)**: Adds a nested sub-filter (useful for grouping, e.g., (A OR B) AND C).

Step 3: Handling Complex Logic (Nested Filters)

Sometimes you need combinations of AND and OR. For example, searching for a user by name OR email, but restricting them to a specific department.

Logic: department = 'IT' AND (name LIKE '%search%' OR email LIKE '%search%')

Implementation:

```
public class StaffFilter implements SqlFilter {
    private String searchJson; // Generic search input
    private String department;

    // ... setters ...

    @Override
```

```
    public Pair<String, List<Object>> toSql() {
        // 1. Start with the main AND group
        GenericSqlFilter root = GenericSqlFilter.and();

        // 2. Add the exact match for department
        root.eq("department", department);

        // 3. Build the OR logic for the search field
        if (searchJson != null && !searchJson.isBlank()) {
            GenericSqlFilter orClause = GenericSqlFilter.or()
                    .like("name", searchJson)
                    .like("email", searchJson);

            // 4. Nest the OR clause into the main AND group
            root.add(orClause);
        }

        return root.toSql();
    }
}
```

3. Integration with Repository

The EntityDbRepository class is designed to work seamlessly with SqlFilter objects. Specifically, the findAllOnPage method accepts a filter and uses it to construct dynamic SQL queries.

How it works in EntityDbRepository:

1. **Count Query:** The repository first executes a SELECT COUNT(*) query to determine the total number of records matching the filter.
   ```
   // Inside count() method
   String sql = "SELECT COUNT(*) as count FROM (" + sqlSelectAllStatement + ") t";
   Pair<String, List<Object>> sqlFilter = filter.toSql();
   if (!sqlFilter.getFirst().isEmpty()) {
       sql += " WHERE " + sqlFilter.getFirst(); // Appends the WHERE clause
   }
   // Parameters from sqlFilter.getSecond() are then set on the PreparedStatement
   ```

2. **Fetch Query:** If the count > 0, it executes the main SELECT query with the same filter logic, plus LIMIT and OFFSET for pagination.
   ```
   // Inside findAllOnPageInternal() method
   ```

```
String sql = sqlSelectAllStatement;
Pair<String, List<Object>> sqlFilter = filter.toSql();
if (!sqlFilter.getFirst().isEmpty()) {
   sql += " WHERE " + sqlFilter.getFirst();
}
sql += " LIMIT ? OFFSET ?";
```

Key Takeaway: Because EntityDbRepository automatically appends the WHERE clause generated by your filter, you only need to focus on defining the *conditions* in your SqlFilter implementation. The repository handles the SQL construction and parameter binding safely.

4. Best Practices

1. **Database Column Names**: The string passed to methods like .eq("client_type", ...) must match the **actual column name in your database table**, not your Java field name.
2. **Null Handling**: GenericSqlFilter automatically ignores any condition where the value is null. You don't need to write if (value != null) checks in your filter class.
3. **UI Binding**: Create a new instance of your Filter class in your Controller. Bind UI inputs (TextFields, ComboBoxes) to the setters of your filter object. When the user clicks "Search", pass this filter object to your Service.

5. Usage in Controller

```
@FXML
public void handleSearch() {
   // 1. Create Filter
   ClientFilter filter = new ClientFilter();

   // 2. Bind UI values
   filter.setNameSearch(nameField.getText());
   filter.setMinBudget(budgetSpinner.getValue());

   // 3. Pass to Service (which delegates to repository)
   // The service/repository will use the filter to generate the WHERE clause
   service.findAllOnPage(pageable, filter).thenAccept(...)
}
```